

Physical-Digital Programming

Jasper Tran O’Leary
jaspero@cs.washington.edu
University of Washington
Seattle, Washington, USA

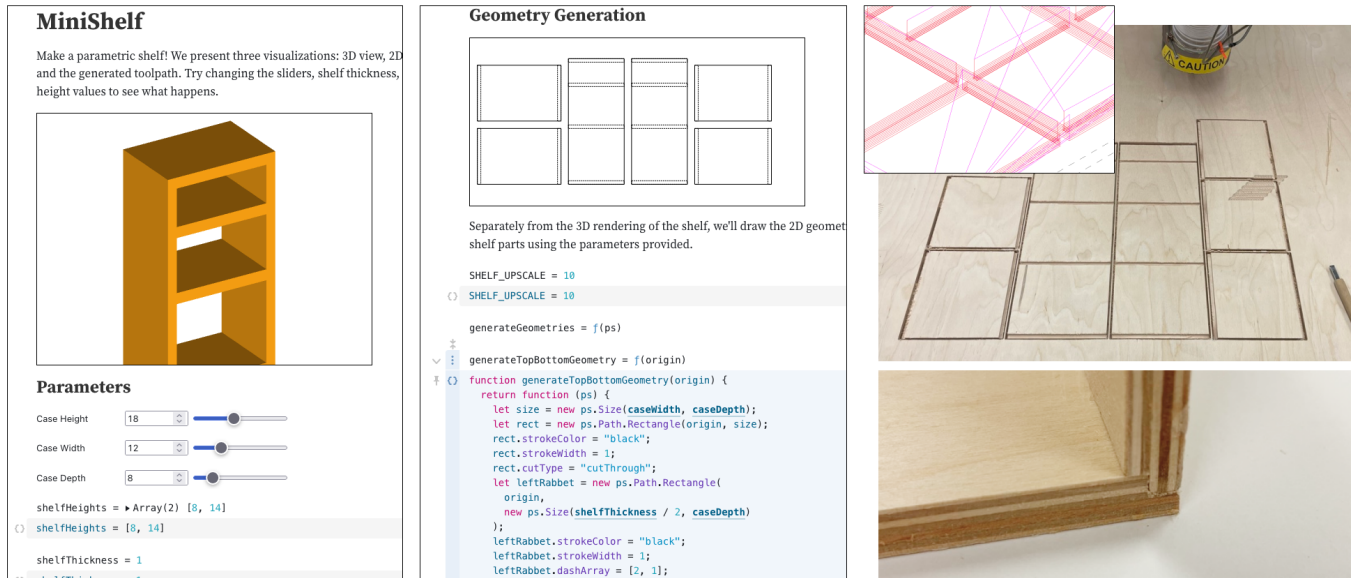


Figure 1: Snippets from an Imprimer notebook. Left: notebook users experiment with shelf dimensions using a 3D rendering that updates immediately with code changes. Middle: the notebook generates new planar shelf geometries as defined in user-authored notebook cells. Right: A generated 3D toolpath (top) that can be directly dispatched to a CNC mill (middle); rabbit joint parameters can be adjusted to achieve a tight fit (bottom).

ABSTRACT

In exploratory digital fabrication, artists, scientists, and engineers use machines and code to prototype object forms and ways of making that do not currently exist. However, prior research has yet to provide expressive agency for makers doing this work, which includes rapid experimentation, safe adjustments and detailed, accurate control of machines and data. To address this, I frame exploratory fabrication as physical-digital programming: handling physical contingencies within core features of a programming language. I posit that physical-digital programming will let exploratory makers build digital fabrication workflows that are otherwise prohibitively difficult to construct, share, and improve. To test this hypothesis, I built three systems that prototype components of physical-digital programming: a common, multimodal programming environment; a formal grammar of machine functionality; and a method of generating task-specific visualizations of machine

behavior. In addition, I am working on a fourth system intended to synchronize code assumptions with physical-world states through user-defined axioms and augmented reality. These tools will help both novice and experienced makers use programming to solve emerging problems that require physical automation.

CCS CONCEPTS

• **Human-centered computing** → **Interactive systems and tools**; • **Applied computing** → **Computer-aided manufacturing**; • **Software and its engineering** → **Domain specific languages**.

KEYWORDS

Digital fabrication, computational notebooks, augmented reality, programming languages

ACM Reference Format:

Jasper Tran O’Leary. 2023. Physical-Digital Programming. In *The 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23 Adjunct)*, October 29–November 01, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3586182.3616709>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST ’23 Adjunct, October 29–November 01, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0096-5/23/10.

<https://doi.org/10.1145/3586182.3616709>

1 INTRODUCTION

Practitioners across domains are increasingly relying on digital fabrication machines to innovate: artists craft sculptures with code, scientists automate hypothesis testing with high-throughput experimentation, and engineers prototype novel material structures with machine precision. These practitioners prioritize *exploratory fabrication*: experimentation, safe adjustments, and expressive control of machines and data. However, current tools for controlling machines are designed primarily for manufacturing workflows that replicate existing objects. End-user fabricators must cobble together novel workflows using ill-fitting GUIs and scripts, resulting in ad hoc programs that are unsafe, divorced from physical concerns, and difficult to edit.

Who Programs? Empirical tests with end-user fabricators from a variety of programming and fabrication backgrounds suggest that programming can broaden access to, and enrich applications for, digital fabrication machines [2, 5]. A prior study [22] showed that drawing machine hobbyists are not only already programming, but they also readily share their code and approaches with others. Further, work with fabrication machines in art, science, and engineering frequently relies on code rather than on conventional CAD and CAM solutions alone. *Rather than avoiding code, I propose to give practitioners and researchers language-level support for the work they are already seeking to accomplish.*

Why Programming? Bringing programming to digital fabrication can help reduce the learning curve for novices by making the different steps in a fabrication workflow explicit in code. However, it also targets expert fabricators working in uncharted domains. Novices and experts alike could benefit from the error-checking and quick experimentation that programming languages provide. Further, *physical-digital programming*, which I describe further in the next section, could introduce programming to manual fabricators who seek to incorporate digital techniques into their craft.

Prior research in human-computer interaction (HCI), computer graphics, and robotics has not yet directly addressed how to provide *expressive agency* for end-users doing exploratory fabrication. Instead, recent HCI work on personal fabrication [1, 3] focuses on building end-to-end systems that accomplish a new task for an end-user. These proof-of-concept pipelines address ease-of-use for novices rather than programmatic control. On the other hand, computer graphics researchers have leveraged a body of work on simulation [23] and optimization [4] to generate and inform new workflows. Human-robot interaction has explored indirect robot programming across environmental contexts [17]; in this case, end-users actively shape and examine robot behavior through programming by demonstration or other means, but the range of programming functionality is constrained and site-specific.

These threads of work are significant because they give researchers and practitioners new approaches to the use of fabrication machines. However, *they do not adequately address the key element that makes digital fabrication truly digital: programming languages.* Unlike exploratory makers, programmers working in digital-only domains rely on features of programming languages that could

offer much to fabricators, including:

Rapid, Safe Experimentation. Digital fabrication tools today have inherited batch programming-style techniques from their manufacturing inception in the 1950s. End-user fabricators must re-run entire multi-program workflows to make changes to their designs. On the other hand, programmers today can rapidly run code, use interactive debuggers, and design tests with testing frameworks.

Error Checking. “Make illegal states unrepresentable” [9] is a guiding principle of functional programming: a programming language should discourage unsafe code through a type system, static analysis, language design, or other approaches. Such techniques have not yet been applied to code that deals with physical systems; instead, end-user fabricators rely on modeling and simulation, a heavyweight approach that is not feasible for initial explorations.

Building Abstractions. Programmers construct libraries for others to use. Designing domain-specific languages (DSLs) can help scaffold computational thinking for a particular application.

For these reasons, I contend that we should view exploratory digital fabrication as *reading and writing programs*, not just executing them. While prior work has explored fabrication concepts at the design stage [6, 11, 24], fewer investigate downstream concerns such as low-level machine control [16] or material-informed iteration [18]. In this case, our programming tools should be fundamentally adapted to handle challenges beyond the computer screen; I will call this concept *physical-digital programming*.

2 PHYSICAL-DIGITAL PROGRAMMING

Physical-digital programming represents physical contingencies (e.g., idiosyncratic machine behaviors, sensor or manual input, and post-fabrication verification of dimensions) as language-level constructs.

A *physical-digital program* comprises transformations of data between: design constraints, geometries, hardware specifications, hardware instructions, quality control, sensor data, on-screen and in situ visualizations, and human-in-the-loop steps, all within a single programming environment.

I hypothesize that *physical-digital programming will let practitioners in art, science, and engineering build exploratory digital fabrication workflows that are otherwise prohibitively difficult to construct, share, and improve.* To test this hypothesis, I built three systems—Imprimer, Taxon, and Toolpath Stylesheets (described below)—that prototype components of physical-digital programming, and I am working on a fourth system that uses augmented reality and user-defined axioms to align physical state with digital code.

3 IMPRIMER: LITERATE PROGRAMMING FOR DIGITAL FABRICATION

To provide a multimodal programming environment for exploratory fabrication, I draw from *literate programming*, where programs are

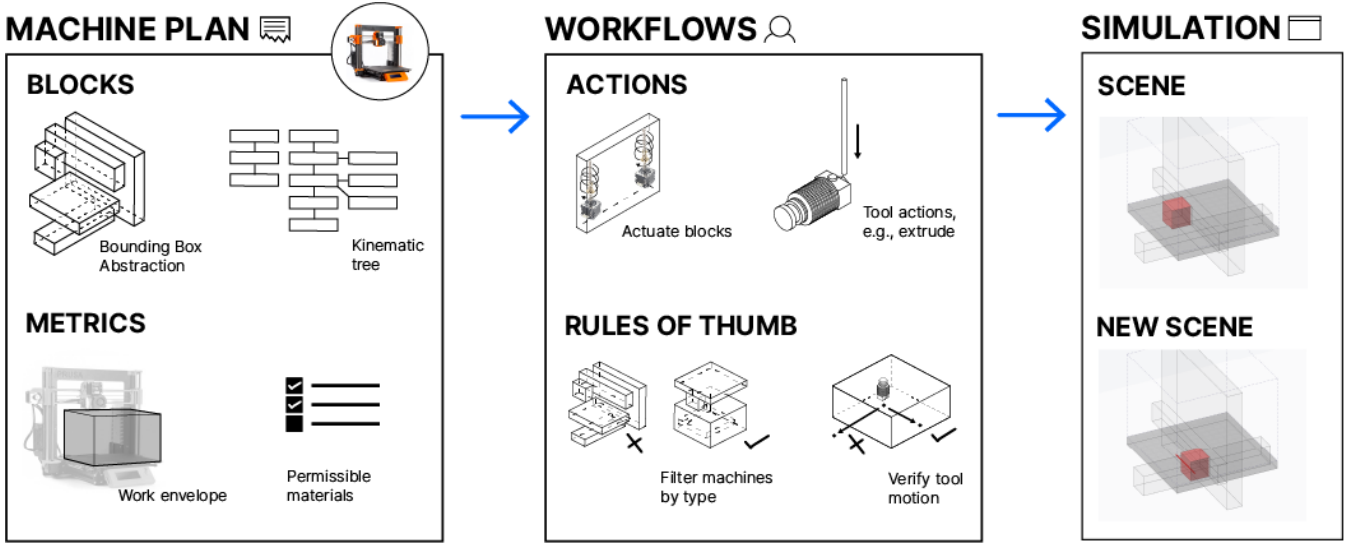


Figure 2: Taxon System Architecture. Makers represent a physical machine as a Taxon Machine Plan program. The program contains: *blocks*, which provide an abstracted composition of the machine, and *metrics*, which describe high-level machine characteristics, such as the volume that its tool can access (work envelope) and material compatibility. Taxon Machine Plans can be used in workflows containing *actions*, which simulate motion and fabrication. *Rules of thumb* are a database of community-contributed rules useful when analyzing the safety of actions or machine plans/actions for specific machines and manufacturing tasks. Executing an action updates the machine in the simulation, in this case moving the tool (red) to the directed position.

human-readable documents that interleave prose, graphics, and source code [8]. Computational notebooks, such as Jupyter Notebook [7] and Observable [12], are coding interfaces for literate programming; they provide assistance with digital-only programming problems in domains like data science and machine learning [13].

To support exploratory programming with both digital data and physical machines and materials, Imprimer [19] provides a *machine infrastructure for structured, direct control of a CNC mill* [15] from a *computational notebook* [12] (Figure 1). Using Imprimer, makers prototype new fabrication workflows by writing and modifying code. The Imprimer library provides custom visualizations both in the notebook and projected onto the machine in situ. Makers document their process by interleaving text and visual documentation.

Imprimer blends traditionally separate parts of the fabrication process into a single, live environment where makers can make changes to code, visualize the results, and deploy cutting jobs immediately to the machine. Makers can also view or hide the underlying code for each cell; by hiding all code, they can use the notebook as they would any other graphical user interface or tutorial document.

4 TAXON: FORMALIZING MACHINE COMPOSITION

To provide an extensible formal foundation for fabrication as programming, Taxon[21] is a *grammar that represents digital fabrication machines as programs, letting users reason formally about what machines can and cannot do* (Figure 2). Taxon brings programming language techniques like static analysis and program synthesis into the world of digital fabrication, e.g., to examine machine workflow code and detect errors such as using cutting speed

settings that are too high for a given material. Inspired by visualization grammars [14], Taxon renders programs into simulations in a browser-based environment so users can visualize and experiment with different machines and workflows before moving to physical machines and materials.

Crucially, Taxon lets programmers represent otherwise-informal knowledge about machine usage as *rules of thumb*. Rules of thumb are implemented as functions that either: take a machine as an argument and filter for machines that satisfy a desired characteristic, or check whether a given line of code is safe or not. Through rules of thumb, fabricators can express best practices as formalized logic that can be shared and run across many fabrication-as-programs, thus extending and enriching Taxon as the capabilities of and applications for machines increase.

5 TOOLPATH STYLESHEETS: UNDERSTANDING MACHINE BEHAVIOR

An important part of authoring novel fabrication workflows is deeply understanding machine behavior. While some of this intuition comes from physically operating machines, as end-users develop increasingly specialized workflows, there is increased need to visualize different parts of the workflow beyond the standardized visualizations that typical CAD/CAM software provides.

To develop bespoke visualizations, toolpath stylesheets (TSSs) [20] generate visualizations from the set of low-level machine instructions that constitute a given toolpath where the machine's tool physically moves. A TSS interprets a toolpath's instructions with a programmer-defined graphical semantics. For example, under the

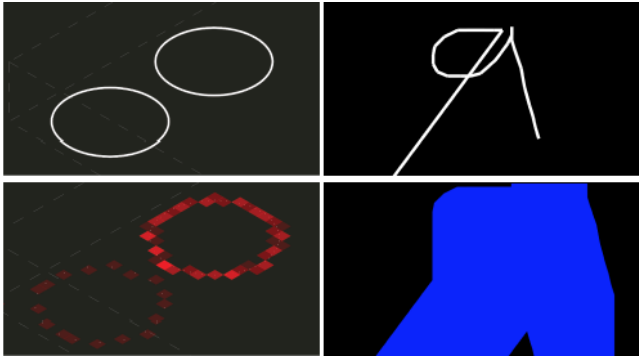


Figure 3: Toolpath Stylesheets (TSS). Left: a laser cutter toolpath with overlapping geometries. A standard toolpath rendering (top) reveals only two apparent circles, but a time-weighted 2D histogram of points (bottom) reveals that the laser spends dangerously high amounts of time on re-cutting one circle. Right: a TSS illustrating the area that the carriage of a CNC router will cover on the cutting bed (bottom) when cutting a given design (top). The former is calculated by taking the convex hull of the bounding box corners of the tool head’s geometry across all instructions of the toolpath.

semantics of a machine’s hardware controller, the G-code instruction `G0 X10 Y15 Z0.10` moves the tool to the position (10, 15, 0.10); instead, under a TSS’s semantics, the instruction might draw a line, change a color, or add a point to a 2D histogram (Figure 3, right).

Using TSS, end-users can rapidly develop visualizations that illustrate important nuanced machine behavior, e.g., the space on a CNC mill bed where tall fixtures cannot be placed without colliding with the machine (Figure 3, left).

6 AXIOMATIZING HUMAN-MACHINE INTERACTION (IN PROGRESS)

My most significant remaining challenge involves synchronizing steps that must be manually undertaken in the physical world with the state of digital code. For example, consider the task of milling a propeller with a common three-axis CNC mill. Because the end mill can only cut from the top downwards, to carve the underside of the propeller blades, a fabricator must flip the workpiece (the piece of material being cut) halfway through the milling process. This requires not only additional design and machine programming, but heavy manual intervention at all stages of the process. First, fabricators must mill a separate jig to hold the workpiece after it is flipped; this alignment jig’s design varies based on the propeller’s dimensions. They must also constantly re-zero the machine’s coordinate axis to specific points relative to the workpiece, change the mill’s cutting tool for each cutting operation, and verify that completed operations cut the workpiece to an acceptable tolerance.

To address this, I am building a system that provides augmented reality (AR) overlays on the machine’s bed to guide fabricators through manual stages of the fabrication process. Overlays can be declaratively specified in the computational notebook interface. In parallel, user-defined *axioms* provide guidance for what successfully completed steps should look like. Any issues that arise in the

physical world prompt re-inspection of code in the notebook environment, which is also linked to an industry-standard CAD/CAM tool: as the user makes adjustments in the notebook, the CAD/CAM interface updates and regenerates changes accordingly. In the reverse direction, changes made in the CAD/CAM tool propagate to the notebook and the AR overlay.

7 FUTURE WORK

7.1 Contributions to Computer Science

Representing exploratory fabrication workflows as computational notebooks has thus far covered only a small sample of possible applications. Many more fabrication challenges remain that demand a deep understanding of more nuanced experimental fabrication workflows. In future work, I aim to apply physical-digital programming techniques to novel challenges in art, science, and engineering, formalizing fresh challenges with physics and materials, data representation and visualization, and human-robot interaction. For example, I will examine robots for non-manufacturing proposes, such as robot arms and robotic lab automation equipment.

These additional formalisms and proof-of-concept physical-digital programs will advance research in programming languages, robotics, and visualization communities. I also plan to develop more algorithms, interaction techniques, and domain-specific languages to better support ideation and building for exploratory fabrication.

7.2 Physical-Digital Programs as Open Science

By authoring physical-digital programs, researchers can archive and share scientific contributions that use digital fabrication machines. Imprimer notebooks can explain *and* implement existing work, such as a novel technique of 3D printing a wireframe of an object [10]. Those interested in this paper need only open the notebook in a browser,¹ experiment with the code, and immediately print their own wireframe prints without the need for dedicated CAD/CAM software. Building free and open-source, remixable, and scaffolded control software for diverse fabrication use cases is the first step towards a future of reproducible and replicable work with machines.

7.3 Dissertation Status

At the time of writing, I am beginning the seventh and last year of my PhD and will be entering the job market in the coming academic year. Critique and discussion from the doctoral symposium panel would be of significant benefit in helping me learn how to best frame my work to appeal to future collaborators across disciplines.

ACKNOWLEDGMENTS

I would like to thank my advisor, Nadya Peek, for her support throughout my PhD in wrestling with these ideas and for encouraging me to become a better maker. My gratitude also extends to the collaborators and mentors who have worked with me on the research projects described in this paper. Finally, I would like to thank the Alfred P. Sloan Foundation for financial support while undertaking this research.

¹Available at: <https://observablehq.com/@jtrano/leary/imprimer-wireprint>

REFERENCES

- [1] Patrick Baudisch and Stefanie Mueller. 2017. Personal Fabrication. *Foundations and Trends® in Human–Computer Interaction* 10, 3–4 (2017), 165–293. <https://doi.org/10.1561/11000000055>
- [2] Frikk H Fossdal, Vinh Nguyen, Rogardt Heldal, Corie L. Cobb, and Nadya Peek. 2023. Vespidae: A Programming Framework for Developing Digital Fabrication Workflows. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference (DIS '23)*. Association for Computing Machinery, New York, NY, USA, 2034–2049. <https://doi.org/10.1145/3563657.3596106>
- [3] Neil Gershenfeld. 2007. *Fab: The Coming Revolution on Your Desktop—from Personal Computers to Personal Fabrication*. Basic Books, Inc., USA.
- [4] Aldair E. Gongora, Siddharth Mysore, Beichen Li, Wan Shou, Wojciech Matusik, Elise F. Morgan, Keith A. Brown, and Emily Whiting. 2021. Designing Composites with Target Effective Young's Modulus using Reinforcement Learning. In *Symposium on Computational Fabrication (SCF '21)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3485114.3485123>
- [5] Jennifer Jacobs and Leah Buechley. 2013. Codeable Objects: Computational Design and Digital Fabrication for Novice Programmers. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '13)*. ACM, New York, NY, USA, 1589–1598. <https://doi.org/10.1145/2470654.2466211>
- [6] Jun Kato and Masataka Goto. 2017. f3.js: A Parametric Design Tool for Physical Computing Devices for Both Interaction Designers and End-users. In *Proceedings of the 2017 Conference on Designing Interactive Systems (DIS '17)*. Association for Computing Machinery, New York, NY, USA, 1099–1110. <https://doi.org/10.1145/3064663.3064681>
- [7] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian E. Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica B. Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, Carol Willing, and Jupyter Development Team. 2016. Jupyter Notebooks - a publishing format for reproducible computational workflows. In *ELPUB*.
- [8] Donald E. Knuth. 1984. Literate Programming. *Comput. J.* 27, 2 (may 1984), 97–111. <https://doi.org/10.1093/comjnl/27.2.97>
- [9] Yaron Minsky. 2010. Effective ML. <https://www.youtube.com/watch?v=J8YyfrSwTk>
- [10] Stefanie Mueller, Sangha Im, Serafima Gurevich, Alexander Teibrich, Lisa Pfisterer, François Guimbretière, and Patrick Baudisch. 2014. WirePrint: 3D Printed Previews for Fast Prototyping. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology (UIST '14)*. ACM, New York, NY, USA, 273–280. <https://doi.org/10.1145/2642918.2647359>
- [11] Chandrakana Nandi, James R. Wilcox, Pavel Panchekha, Taylor Blau, Dan Grossman, and Zachary Tatlock. 2018. Functional Programming for Compiling and Decompiling Computer-aided Design. *Proc. ACM Program. Lang.* 2, 1ICFP (July 2018), 99:1–99:31. <https://doi.org/10.1145/3236794>
- [12] Observable. 2022. *Observable HQ*. <https://observablehq.com/>
- [13] Jeffrey M. Perkel. 2021. Reactive, reproducible, collaborative: computational notebooks evolve. *Nature* 593 (2021), 156–157. <https://doi.org/10.1038/d41586-021-01174-w>
- [14] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2017. Vega-Lite: A Grammar of Interactive Graphics. *IEEE Transactions on Visualization and Computer Graphics* 23, 1 (Jan. 2017), 341–350. <https://doi.org/10.1109/TVCG.2016.2599030>
- [15] ShopBot. 2022. *ShopBot Tools*. <https://www.shopbottools.com/>
- [16] Blair Subbaraman and Nadya Peek. 2022. p5.fab: Direct Control of Digital Fabrication Machines from a Creative Coding Environment. In *Proceedings of the 2022 ACM Designing Interactive Systems Conference (DIS '22)*. Association for Computing Machinery, New York, NY, USA, 1148–1161. <https://doi.org/10.1145/3532106.3533496>
- [17] Ryo Suzuki, Adnan Karim, Tian Xia, Hooman Hedayati, and Nicolai Marquardt. 2022. Augmented Reality and Robotics: A Survey and Taxonomy for AR-enhanced Human-Robot Interaction and Robotic Interfaces. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*. Association for Computing Machinery, New York, NY, USA, 1–33. <https://doi.org/10.1145/3491102.3517719>
- [18] Iremnur Tokac, J. Philips, H. Bruyninckx, and A. Vande Moere. 2021. Fabrication grammars: bridging design and robotics to control emergent material expressions. *Construction Robotics* (2021). <https://doi.org/10.1007/s41693-021-00053-0>
- [19] Jasper Tran O'Leary, Gabrielle Benabdallah, and Nadya Peek. 2023. Imprimer: Computational Notebooks for CNC Milling. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. Association for Computing Machinery, New York, NY, USA, 1–15. <https://doi.org/10.1145/3544548.3581334>
- [20] Jasper Tran O'Leary, Eunice Jun, and Nadya Peek. 2022. Improving Programming for Exploratory Digital Fabrication with Inline Machine Control and Styled Toolpath Visualizations. In *Proceedings of the 7th Annual ACM Symposium on Computational Fabrication (SCF '22)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3559400.3561998>
- [21] Jasper Tran O'Leary, Chandrakana Nandi, Khang Lee, and Nadya Peek. 2021. Taxon: a Language for Formal Reasoning with Digital Fabrication Machines. In *The 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21)*. Association for Computing Machinery, New York, NY, USA, 691–709. <https://doi.org/10.1145/3472749.3474779>
- [22] Hannah Twigg-Smith, Jasper Tran O'Leary, and Nadya Peek. 2021. Tools, Tricks, and Hacks: Exploring Novel Digital Fabrication Workflows on #PlotterTwitter. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (Yokohama, Japan) (CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 594, 15 pages. <https://doi.org/10.1145/3411764.3445653>
- [23] Nobuyuki Umetani, Danny M. Kaufman, Takeo Igarashi, and Eitan Grinspun. 2011. Sensitive Couture for Interactive Garment Modeling and Editing. In *ACM SIGGRAPH 2011 Papers (SIGGRAPH '11)*. ACM, New York, NY, USA, 90:1–90:12. <https://doi.org/10.1145/1964921.1964985> event-place: Vancouver, British Columbia, Canada.
- [24] Chenming Wu, Haisen Zhao, Chandrakana Nandi, Jeffrey I. Lipton, Zachary Tatlock, and Adriana Schulz. 2019. Carpentry Compiler. *ACM Trans. Graph.* 38, 6 (Nov. 2019), 195:1–195:14. <https://doi.org/10.1145/3355089.3356518>